UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

FACULTAD DE INGENIERIA
en acción continua...

ALGORITMOS Y ESTRUCTURA DE DATOS II

PROYECTO INTEGRADOR

ALUMNO: Leonel Chadi, Martin Merlo, Laureano Petri y Exequiel Mayorga

LEGAJO: 14507, 14335,14571, 14948

AÑO: 2026

REPOSITORIO DEL PROYECTO EN GITHUB:
https://github.com/exespartan32/Algoritmos2_proyecto_final

Capítulo 1: Creación del Grafo y de escenarios

Este capítulo describe la capa sobre la que trabaja todo el resto del sistema: la lectura y validación del escenario en JSON, la construcción del dígrafo de pasillos del almacén, el cálculo de las celdas de acceso a racks y zonas de despacho, la verificación de transitabilidad mediante componentes fuertemente conexas (Kosaraju) y el generador determinista de escenarios. Los algoritmos se explican como una secuencia de pasos, acompañados de ejemplos obtenidos al ejecutar el código del proyecto.

1.1 Objetivo del módulo

El armado de waves, el cálculo de rutas y la simulación necesitan responder constantemente preguntas como «¿a qué celdas puede moverse un picker desde acá?» o «¿en qué celda tiene que frenar para retirar este producto?». El módulo de modelado existe para que esas preguntas se respondan en tiempo constante y para garantizar, antes de simular, que el mapa es transitable. Tiene cuatro responsabilidades:

1. Revisar que el escenario tenga sentido y rechazarlo con un mensaje claro si no lo tiene.
2. Traducir los rectángulos del JSON a un dígrafo de celdas que respete el sentido de circulación de cada pasillo.
3. Precalcular desde qué celda de pasillo se retira cada producto y desde cuáles se descarga en cada zona.
4. Controlar que no existan trampas (zonas de las que no se puede salir) ni islas (zonas a las que no se puede llegar), y avisar si las hay.

Archivos involucrados: `validador_escenario.py` (validación), `graph_builder.py` (construcción y control del grafo), `warehouse.py` (almacén y dígrafo), `entities.py` (celdas, racks, productos, pedidos, pickers) y `scenario_generator.py` (generador).

1.2 El escenario en JSON

Un escenario describe el almacén mediante rectángulos: cada elemento indica la celda de su esquina inferior izquierda (`ubicacion`), su `ancho` y su `alto`. El eje y crece hacia arriba, y la celda (0, 0) es la de abajo a la izquierda.

Sección	Qué trae	Qué se hace con eso
`dimensiones`	`ancho_x`, `largo_y`	Define la grilla de celdas.
`pasillos`	id, rectángulo y `sentido` (N, S, E, O o AMBOS)	Define el sentido de circulación de cada celda que cubre.
`racks`	id y rectángulo (un lado debe medir 2)	Sus celdas son obstáculos y guardan los productos por nivel.

`zonas_despacho`	id, rectángulo, puerta asociada y `peso_max_kg`	Sus celdas son obstáculos; se calculan sus celdas de acceso.
`puertas`	id, tipo (INBOUND, OUTBOUND, AMBOS) y una celda	Obstáculo en el borde del almacén.
`productos`	id, nombre, rack, posición (x, y, z), stock y peso	Van al catálogo y a su celda de rack, en el nivel z.
`pedidos`	id, prioridad (1 a 5), estado e ítems	Se cargan para el armado de waves.
`pickers`	id, posición inicial y capacidad en kg	Se ubican sobre una celda de pasillo.
`configuracion`	máximo de pedidos y de productos por wave	Límites del armado de waves (por defecto, 3 y 15).

Las aristas del grafo no vienen en el JSON: se deducen a partir de la grilla y de los sentidos de los pasillos**.**

1.3 Representación: el dígrafo de pasillos

Elementos del modelo

- Nodo: toda celda que no es obstáculo (rack, zona o puerta). Si ningún pasillo la declara, es piso libre y queda con sentido AMBOS.
- Arista dirigida: «desde esta celda se puede ir a esa otra en un paso», con costo 1. Es dirigida porque, en un pasillo de mano única, puede existir la arista de A a B sin que exista la de B a A.
- Acceso: celda de pasillo donde el picker frena para retirar un producto de un rack o para descargar en una zona.
- Altura: el picker se mueve siempre en el piso (z = 0). La altura existe solo dentro de cada celda de rack, que guarda qué producto hay en cada nivel z = 1, 2, …

Estructuras de datos

Estructura	Clave → valor	Uso
`nodos`	(x, y) → datos de la celda (sentido, pasillo, racks y zonas vecinas)	Saber si una celda es transitable y con qué sentido.
`lista_adyacencia`	(x, y) → lista de celdas alcanzables en un paso	Las aristas del dígrafo.
`obstaculos`	(x, y) → celda de rack, zona o puerta	Saber qué hay en una celda no transitable.

`productos_en_z`	nivel z → producto (dentro de cada celda de rack)	Modelar la altura.
`acceso_rack_a_pasillo`	celda de rack → celda de pasillo	Dónde frena el picker, en O(1).
`accesos_zona_a_pasillo`	id de zona → celdas de pasillo vecinas	Dónde puede descargar.
`catalogo_productos`, `pedidos`	id → objeto	Búsqueda por id en O(1).

¿Por qué un dígrafo? Porque existen pasillos de mano única, que un grafo no dirigido no puede representar.

¿Por qué lista de adyacencia y no matriz? Cada celda tiene a lo sumo cuatro vecinos, así que el grafo es ralo ($E \leq 4V$). La lista ocupa $O(V + E)$. Una matriz ocuparía $O(V^2)$: en el escenario grande, con 1898 nodos, serían más de 3,6 millones de casilleros para solo 6048 aristas. Además, obtener los vecinos de una celda, la operación que más usan las rutas, es inmediato.

¿Por qué indexar por coordenada? La coordenada (x, y) es la identidad natural de una celda: así aparece en el JSON y así la selecciona el usuario en la interfaz. Las estructuras son tablas hash indexadas por coordenada, de modo que se accede a cualquier celda en tiempo constante promedio, sin búsquedas.

1.4 Validación del escenario

Antes de construir nada, `validar_escenario` revisa el JSON. Si encuentra un problema, se produce un error: se muestra un mensaje y no se construye el almacén. Los pasos son:

1. Revisa tipos y rangos: ancho y largo enteros de 1 a 200, coordenadas enteras de 0 a 199, ids de texto no vacíos, pesos y capacidades mayores que cero, nivel z desde 1, cantidades desde 1, prioridad de 1 a 5. Un campo obligatorio faltante también es un error.
2. Revisa que no haya ids repetidos entre racks, puertas, zonas y pasillos.
3. Revisa que cada rectángulo esté dentro del mapa y que racks, puertas y zonas no se superpongan entre sí. Los pasillos sí pueden superponerse.
4. Revisa las reglas de cada elemento: el rack tiene un lado de 2; la puerta está en el borde y su tipo es válido; el sentido del pasillo es N, S, E, O o AMBOS; la zona es adyacente a su puerta.
5. Revisa los productos: cada uno está en una celda de su propio rack, y no se repiten ids ni posiciones (x, y, z).
6. Revisa los pedidos: id único, al menos un ítem, productos existentes, cantidades positivas y sin el mismo producto dos veces.
7. Revisa los pickers: al menos uno, sin ids ni posiciones repetidas, dentro del mapa y no sobre un obstáculo.

8. Revisa que exista al menos una zona de despacho con puerta de salida (OUTBOUND o AMBOS).

Esta validación no comprueba la conectividad de los pasillos, porque para eso hace falta el grafo construido. Esa comprobación se hace con Kosaraju (sección 1.7).

1.5 Algoritmo de construcción del almacén

Lo realiza `GraphBuilder.construir_warehouse_desde_dict`. Durante la construcción se llevan dos registros auxiliares: qué elemento ocupa cada celda (para avisar si dos elementos se superponen) y qué sentido declaró el JSON para cada celda de pasillo.

1. Crea el almacén vacío con las dimensiones del JSON.
2. Registra los pasillos. Recorre cada celda de cada pasillo y anota que ese pasillo la ocupa. Si otro elemento ya la ocupaba, guarda la advertencia «X pisa a Y en (x, y)». Luego anota el sentido con la regla del cruce: si la celda ya tenía AMBOS, lo conserva; si no, toma el sentido de este pasillo. Así, en un cruce donde alguno de los pasillos es de doble mano, la celda queda de doble mano y se puede girar.
3. Marca las puertas como obstáculos.
4. Marca las zonas de despacho: cada celda de su rectángulo es un obstáculo.
5. Marca los racks: cada celda de su rectángulo es una celda de rack con sus niveles vacíos.
6. Ubica los productos: los guarda en el catálogo por id y los apoya en el nivel z de su celda de rack (z = 1 si no se indica).
7. Crea los nodos: recorre toda la grilla; toda celda que no sea obstáculo pasa a ser un nodo, con el sentido anotado o, si no tiene, con AMBOS.
8. Calcula los accesos y las aristas de cada nodo (secciones 1.6 y 1.7).
9. Ejecuta Kosaraju y la validación de transitabilidad (sección 1.8).
10. Carga la configuración, los pedidos y los pickers. Si un picker comienza sobre una celda de rack, se lo reubica en su acceso; si comienza sobre una zona, en el acceso más cercano de esa zona.

Como los obstáculos se marcan después de los pasillos, si un rack o una zona quedan encima de un pasillo, esa celda queda como obstáculo y se informa la superposición.

1.6 Regla de las aristas

Para cada nodo de origen se intenta un movimiento hacia arriba, abajo, izquierda y derecha. La arista existe solo si se cumplen tres condiciones:

1. El sentido del origen permite salir en esa dirección. Un pasillo N permite avanzar hacia arriba y doblar a izquierda o derecha, pero no retroceder hacia abajo. Análogamente para S, E y O. AMBOS permite las cuatro direcciones. Las salidas laterales permiten abandonar el pasillo hacia un cruce.
2. El destino es un nodo: está dentro del mapa y no es un obstáculo.
3. No se entra al destino de contramano: no se puede subir a una celda S, bajar a una N, moverse a la derecha hacia una O ni a la izquierda hacia una E.

Ejemplo. Se ejecutó el algoritmo sobre un almacén de 6 × 5 con un pasillo que sube en x = 1, uno que baja en x = 4, dos pasillos de doble mano en y = 0 e y = 4, un rack de 2 × 3, una zona de 1 × 2 y su puerta. Resultan 30 celdas, 9 obstáculos, 21 nodos y 42 aristas.

- Desde (1, 1), sentido N: hacia arriba (1, 2) sí; hacia la izquierda (0, 1) sí; hacia la derecha hay un rack. Resultan dos aristas.
- Desde (4, 1), sentido S: solo puede bajar a (4, 0); a sus lados están el rack y la zona.
- Desde (4, 0), sentido AMBOS: por su propio sentido podría subir, pero (4, 1) es S y subir sería entrar de contramano. No hay arista hacia arriba. Este caso muestra por qué hacen falta tanto la regla de salida como la de entrada.

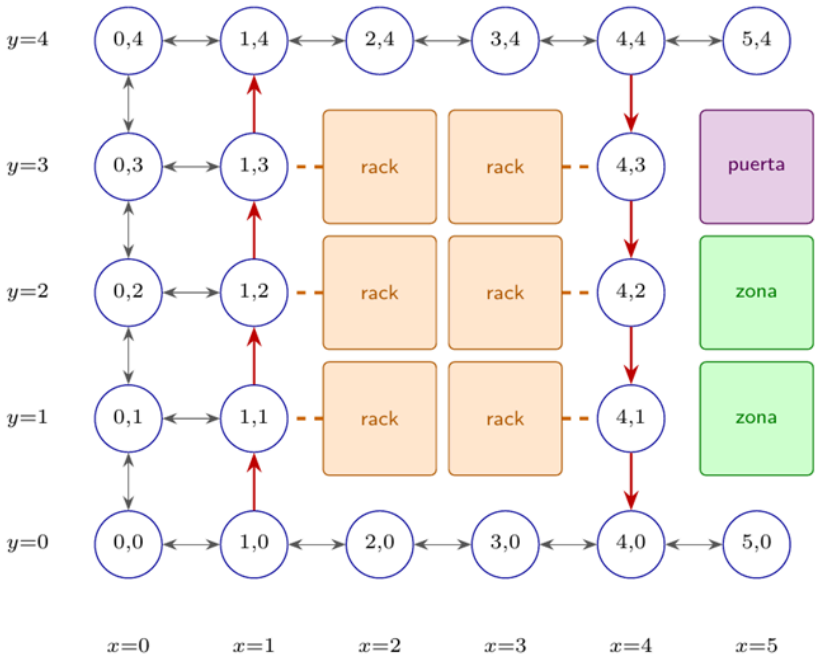


Figura 1.1: Dígrafo del ejemplo. Gris: ida y vuelta. Rojo: una sola mano. Naranja punteado: celda de rack y su acceso.

1.7 Accesos a racks y zonas de despacho

Los racks se operan por sus costados largos, nunca por las puntas. Para cada nodo, se miran sus cuatro vecinos:

- Si el vecino es una celda de un rack vertical (ancho 2), la columna izquierda se accede desde la izquierda y la derecha desde la derecha.
- Si es de un rack horizontal (alto 2), la fila inferior se accede desde abajo y la superior desde arriba.
- Si el rack es de 2 × 2, se accede desde cualquier lado.
- Si el vecino es una celda de una zona de despacho, el nodo queda como acceso de esa zona.

El primer acceso encontrado queda como acceso principal de la celda de rack. Luego la validación lo reemplaza, si hace falta, por uno que esté en la red principal. En el ejemplo, la columna izquierda del rack se accede desde x = 1 y la derecha desde x = 4. La celda (2, 0)

toca el rack por la punta, así que no es acceso. Con esta precomputación, saber dónde frenar para un producto cuesta dos consultas O(1): catálogo → celda de rack → acceso.

1.8 Validación de transitabilidad: Kosaraju

Con pasillos de mano única pueden aparecer trampas e islas. Queremos que todo el almacén forme un único grupo en el que desde cualquier celda se pueda llegar a cualquier otra y volver: una sola componente fuertemente conexa (CFC). El algoritmo de Kosaraju encuentra todas las CFC con solo dos recorridos en profundidad:

1. Primer recorrido (sobre el grafo original). Comienza en una celda y avanza en profundidad hacia celdas no visitadas. Cuando una celda ya no tiene adónde avanzar, se la da por terminada y se la agrega a una lista. Si quedan celdas sin visitar, se vuelve a comenzar desde ellas. El resultado es el orden de terminación de todas las celdas.
2. Transposición. Se construye el grafo transpuesto, invirtiendo todas las aristas.
3. Segundo recorrido (sobre el transpuesto). Se recorre la lista de terminación al revés, empezando por la última celda que terminó. Desde cada celda que todavía no tiene grupo, se juntan todas las celdas alcanzables en el transpuesto que tampoco tengan grupo: ese conjunto es una CFC.
4. Las CFC se ordenan por tamaño. La más grande es la red principal.

Ejemplo. Grafo con aristas A→B, B→C, C→A, C→D, D→E y E→D. El primer recorrido, desde A, termina las celdas en el orden E, D, C, B, A. En el transpuesto, el recorrido desde A (la última en terminar) alcanza solo C y B, porque la arista original C→D ahora es D→C y entra al grupo en lugar de salir. Primer grupo: {A, B, C}. Luego, desde D se alcanza E. La celda C ya tiene grupo, así que no se toma. Segundo grupo: {D, E}.

¿Por qué funciona? La celda que termina última pertenece a un grupo «de arriba», del que se puede salir hacia otros pero al que no se entra desde ellos. Al invertir las aristas, ese grupo queda sin salidas, así que el segundo recorrido junta exactamente ese grupo. Que desde E se pueda llegar a A en el transpuesto no afecta el resultado, por dos motivos: el recorrido nunca comienza por E antes que por A, y cuando le toca a D o a E, las celdas A, B y C ya tienen grupo y se saltean.

Detalles de implementación. Ambos recorridos usan una pila explícita en lugar de recursión, porque Python limita la recursión a unas mil llamadas y un almacén de 200 × 200 puede tener caminos mucho más largos. Se usa búsqueda en profundidad y no en amplitud porque el método necesita el orden de terminación, que solo produce la búsqueda en profundidad.

Advertencias que genera la validación

- Si hay más de una CFC: cuántas son, cuántas celdas quedan fuera de la red principal y en qué zona del mapa.
- Productos sin acceso, o sin acceso dentro de la red principal.
- Zonas sin pasillos adyacentes, o sin pasillo dentro de la red principal.
- Pickers que comienzan fuera de un pasillo o fuera de la red principal.

Además, el acceso principal de cada celda de rack se reemplaza por uno que esté en la red principal, si existe. Estas advertencias no impiden construir el almacén. Por ejemplo, si en el

almacén de la Figura 1.1 todas las columnas se declaran de bajada, Kosaraju encuentra 8 CFC y la validación avisa que 15 celdas, los dos productos, la zona y el picker quedan fuera de la red principal. Durante el desarrollo, este control detectó una trampa real de 41 celdas en `escenario_grande.json`, que luego se corrigió.

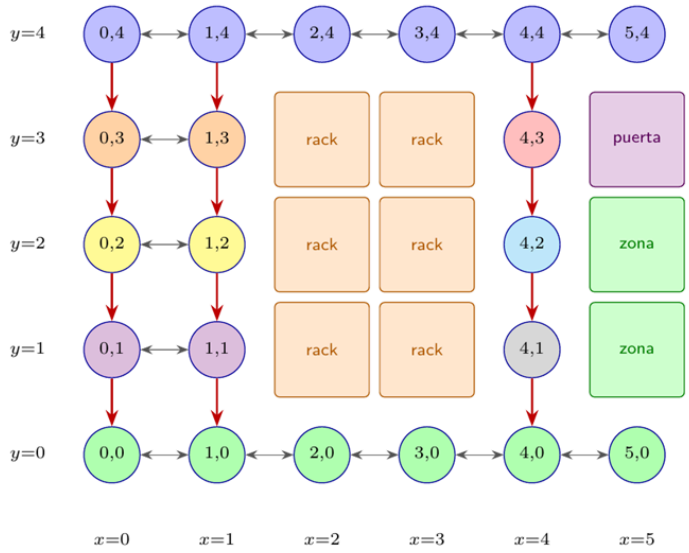


Figura 1.2: Escenario con trampa. Cada color es una componente fuertemente conexa; no existe ninguna arista hacia arriba.

1.9 Generador determinista de escenarios

`ScenarioGenerator` construye escenarios válidos a partir de parámetros: cantidad de estantes, pasillos, productos, pedidos y pickers, máximo de productos por pedido, límites de las waves y localización de los productos. No usa números aleatorios: con los mismos parámetros produce siempre el mismo JSON, lo que permite repetir y comparar experimentos. Los pasos son:

1. Valida los parámetros contra sus rangos (por ejemplo, de 2 a 40 pasillos y de 1 a 400 estantes) y verifica que la capacidad de productos de la wave sea al menos el máximo de productos por pedido.
2. Calcula el tamaño: entre cada par de pasillos verticales hay una columna de racks, así que hay (pasillos − 1) columnas y las filas necesarias para ubicar todos los estantes. Si el almacén resultante supera 200 × 200, se rechaza.
3. Ubica puertas y zonas abajo: una o dos puertas de salida en y = 0, según el ancho, cada una con una zona de 5 × 2 encima.
4. Ubica pasillos y racks: un pasillo transversal de doble mano sobre las zonas (donde comienzan los pickers) y, por cada fila, pasillos verticales de ancho 2 que alternan N y S, con racks de 2 × 6 entre ellos y otro pasillo transversal de doble mano arriba. Los pasillos no se superponen, así que no se generan advertencias de cruce.
5. Ubica los productos según la localización: DISTRIBUIDA intercala los racks para repartirlos parejo; CERCA_DESPACHO llena primero las celdas más cercanas a

las zonas; POR_RACK llena un rack por vez. Si una celda recibe más de un producto, se apilan en niveles z sucesivos.

6. Arma los pedidos: cada pedido tiene entre 1 y el máximo de productos, elegidos con un salto coprimo con la cantidad de productos, de modo que no se repitan dentro de un pedido. La prioridad varía de 1 a 5. El stock de cada producto se fija en su demanda total más un margen, así todos los pedidos son abastecibles.
7. Ajusta capacidades: cada zona soporta el peso total de los pedidos (mínimo 500 kg) y cada picker 1,5 veces el pedido más pesado (mínimo 150 kg). Los pickers se reparten sobre el pasillo transversal inferior.
8. Verifica el resultado con el mismo validador y el mismo `GraphBuilder`. Si aparece cualquier advertencia, el escenario se rechaza. Así, todo escenario generado tiene una única CFC, con todos sus productos, zonas y pickers dentro.

Parámetros	Tamaño	Nodos	Aristas	CFC	Tiempo
Por defecto (12 estantes, 5 pasillos)	18 × 29	356	900	1	16 ms
24 estantes, 7 pasillos, 200 productos	26 × 37	652	1678	1	22 ms
120 estantes, 11 pasillos, 600 productos	42 × 101	2780	7038	1	134 ms

1.10 Comportamiento ante escenarios inválidos

Se probó el sistema con más de 50 escenarios alterados a propósito. La aplicación nunca se cierra por un escenario incorrecto: cualquier error se muestra como mensaje. Al abrir un escenario de la biblioteca que tiene advertencias, se carga y se muestran las advertencias. En cambio, al importar un archivo nuevo se exige que no tenga ninguna advertencia, para que a la biblioteca solo ingresen mapas limpios.

Caso	Resultado	Motivo / mensaje
JSON mal escrito o sin una sección obligatoria	Error	«JSON inválido» o «Formato de escenario inválido (campo faltante…)».
Dimensiones 0, negativas, mayores a 200 o no enteras	Error	«Se esperaba un entero dentro del rango permitido».
Elemento fuera del mapa	Error	«Elemento fuera del mapa».
Rack sin ningún lado de 2	Error	«Un lado del rack debe medir 2».
Racks, zonas o puertas superpuestos; ids repetidos	Error	«Elementos superpuestos» / «Identificador repetido».
Puerta fuera del borde, tipo inválido o sin puerta de salida	Error	Mensaje específico de cada regla.

Zona no adyacente a su puerta	Error	«La zona debe ser adyacente a su puerta».
Sentido inexistente o en minúscula	Error	«Sentido inválido».
Producto fuera de su rack, posición duplicada o z < 1	Error	«Producto fuera de su rack» / «duplicados».
Pedido con producto inexistente, sin ítems, prioridad fuera de 1 a 5 o ítem repetido	Error	Mensaje específico de cada regla.
Sin pickers, picker sobre un rack, fuera del mapa o en una posición repetida	Error	«Se necesita al menos un picker» / «Picker inválido».
Pasillos que se cruzan	Advertencia	El cruce se construye bien, pero se informa la superposición. Se puede abrir, pero no importar.
Pasillo sobre un rack	Advertencia	El rack prevalece: la celda queda como obstáculo.
Todos los pasillos de mano única en el mismo sentido	Advertencia	Kosaraju encuentra varias CFC; se informa qué quedó fuera de la red principal.
Rack con un costado bloqueado	Advertencia	«El producto … no tiene pasillo de acceso».
Sin pasillos declarados	Carga normal	Todo el piso libre es de doble mano.
Dos productos en la misma celda y distinto nivel	Carga normal	Es el modelo de altura.
Stock 0, o pedido mayor que el stock	Carga normal	Lo resuelve el armado de waves.
Sin `configuracion` o con campos extra	Carga normal	Se usan valores por defecto; los campos extra se ignoran.

1.11 Complejidad y resultados

Etapa	Costo	Justificación
Validación del JSON	Lineal en el área y los datos	Cada celda de cada rectángulo y cada dato se revisan una vez.
Marcar pasillos y obstáculos	Θ(área)	Una operación O(1) por celda.

Crear nodos	Θ(área)	Un recorrido de la grilla.
Accesos y aristas	Θ(V)	Cuatro vecinos por nodo.
Kosaraju	Θ(V + E) = Θ(V)	Dos recorridos; E ≤ 4V.
Validación de transitabilidad	Lineal	Recorre accesos, productos, zonas y pickers.
Construcción completa	Lineal en el tamaño del escenario	Ninguna etapa compara todos contra todos.
Vecinos de una celda / acceso de un producto	O(1)	Consulta directa por coordenada o por id.

Escenario	Tamaño	Nodos	Aristas	CFC	Construcción
escenario_basico	18 × 16	230	692	1	6 ms
escenario_gen_mediano	26 × 37	652	1678	1	18 ms
escenario_grande	60 × 40	1898	6048	1	56 ms
escenario_gen_grande	42 × 85	2396	6226	1	86 ms

En mapas de prueba de n × n, al multiplicar el área por 64 (de 25 × 25 a 200 × 200) el tiempo de construcción se multiplicó por 83, y el caso más grande tarda menos de un segundo. El crecimiento es prácticamente proporcional al área, lo que confirma el costo lineal.

Capítulo 2: Creación de estrategias de recorrido

1. Arquitectura del Módulo de Ruteo y Estructuras de Datos Propias

Una vez que una wave ha sido conformada y asignada a uno o varios operarios, el módulo Pathfinding (path_finding.py) se encarga de determinar el recorrido exacto celda a celda dentro del almacén. Todo recorrido válido debe iniciar en la posición actual del picker (o en la zona de despacho), visitar las celdas de pasillo adyacentes a cada rack donde se alojan los productos solicitados y finalizar en una celda de pasillo ortogonalmente adyacente a la zona de despacho asignada, circulando exclusivamente por pasillos habilitados y respetando el sentido de circulación (N, S, E, O, AMBOS).

Para garantizar el cumplimiento de las restricciones académicas sin depender de bibliotecas externas ni heurísticas no estudiadas en la materia, implementamos las siguientes estructuras y componentes fundamentales:

- **Cola de Prioridad Mínima Propia (_MinHeapRuteo):** Implementamos desde cero un montículo binario de mínimos que almacena tuplas (distancia, giros, coordenada). Esta estructura permite insertar nuevos estados y extraer el vértice no visitado de menor distancia acumulada en tiempo logarítmico $O(\log V)$.
- **Desempate por Menor Cantidad de Giros:** En una grilla ortogonal con pasillos de ancho ≥ 2, existen múltiples caminos de igual distancia mínima entre dos puntos (por ejemplo, desplazarse en "zig-zag" cambiando de carril en cada paso tiene el mismo costo que avanzar en línea recta y doblar una sola vez en la esquina). Al incorporar los giros acumulados como segundo criterio de ordenamiento en _MinHeapRuteo (penalizacion_giro), el algoritmo prefiere conservar la dirección de avance actual. Esto produce trayectorias rectilíneas que reducen los cruces laterales entre operarios.
- **Objeto de Transferencia y Métricas (RutaResultado):** Cada ejecución de ruteo encapsula sus resultados en una estructura unificada que registra el camino_completo (secuencia de coordenadas (x, y)), la distancia_total y aristas_recorridas ($\max(0, \vert{}\text{camino}\vert{} - 1)$), el conjunto de nodos_explorados únicos, el total de expansiones del montículo, el orden_exploracion cronológico (utilizado por la interfaz gráfica para animar cómo el algoritmo explora el almacén) y el tiempo_ejecucion_ms.

Propiedad de Optimalidad de Dijkstra en Dígrafos de Costo No Negativo: El algoritmo Dijkstra resuelve el problema del camino más corto desde un vértice origen en grafos dirigidos o no dirigidos con pesos de arista no negativos ($w(u, v) \geq 0$). Mantiene un conjunto S de vértices cuya distancia mínima desde el origen ya es definitiva. En cada paso, extrae del Min-Heap el vértice $u \notin S$ con menor estimación distancias[u], lo incorpora a S y relaja sus aristas salientes. Como todos los movimientos ortogonales entre celdas de pasillo adyacentes tienen costo unitario ($w = 1 > 0$), el primer instante en que un nodo es extraído del montículo garantiza matemáticamente haber hallado el camino mínimo hacia él respetando el sentido de las calles.

2. Traducción de Ubicaciones de Racks a Pasillos y Determinación de Distancias

Los productos se almacenan físicamente dentro de celdas de tipo RACK en coordenadas tridimensionales (x, y, z), las cuales constituyen obstáculos no transitables para los operarios. Por lo tanto, el primer paso antes de ejecutar cualquier búsqueda consiste en proyectar cada ubicación de producto o zona de despacho hacia su celda de acceso válida en la red de pasillos mediante el método obtener_celda_acceso_pasillo:

1. **Consulta en Tabla Hash** $\Theta(1)$**:** Durante la construcción del almacén (GraphBuilder), para cada celda de rack (r_x, r_y) Se identifica qué cara del estante de ancho o alto 2 da hacia un pasillo transitable dentro de la componente fuertemente conexa principal, almacenando el mapeo en el diccionario acceso_rack_a_pasillo[(rx, ry)].
2. **Resolución Dinámica de Respaldo:** Si se consulta una coordenada que aún no está mapeada, el método verifica en $\Theta(1)$ si la coordenada ya pertenece a grafo.nodos o inspecciona sus 4 vecinos ortogonales $((r_x \pm 1, r_y), (r_x, r_y \pm 1))$ para retornar la celda de pasillo adyacente.

De esta manera, la distancia entre dos ubicaciones cualesquiera dentro de una wave se mide como la longitud del camino dirigido más corto sobre el dígrafo de pasillos $\delta_{dígrafo}(u \to v)$, donde cada desplazamiento ortogonal válido aporta costo 1.

3. Estrategias para Ordenar y Recorrer las Ubicaciones de una Wave (Preguntas 3, 4 y 9)

Para preparar una wave, el operario debe partir desde su posición inicial (o acceso a la zona de despacho), recolectar todos los artículos requeridos y regresar a la zona de despacho asignada. Implementamos y contrastamos dos estrategias algorítmicas deterministas basadas en el motor de Dijkstra:

3.1. Estrategia A: Orden Estricto de los Pedidos (calcular_estrategia_orden_pedidos)

En esta primera estrategia, las ubicaciones de recolección se visitan respetando estrictamente la secuencia en que aparecen los pedidos dentro de la wave y sus respectivos ítems:

- Sea D = [d_1, d_2, … , d_k] la lista ordenada de celdas de acceso a los productos requeridos y $d_{retorno}$ la celda de acceso a la zona de despacho.
- El algoritmo mantiene la posición actual del operario p_{actual} e invoca iterativamente a calcular_ruta_dijkstra(p_actual, d_i, grafo) para cada parada $d_i \in D$.
- Si dos productos consecutivos comparten la misma celda de acceso en el pasillo ($d_i = p_{actual}$, por ejemplo, dos productos apilados a distintas alturas z_1 y z_2 en el mismo estante), el algoritmo evita ejecutar una búsqueda redundante y recoge ambos en el mismo punto.
- Al finalizar cada tramo, se concatena ruta_tramo al recorrido total (omitiendo el primer nodo del tramo para no duplicar la celda de empalme) y, tras visitar d_k, se ejecuta un último llamado a Dijkstra desde d_k hasta $d_{retorno}$.

3.2. Estrategia B: Cercanía mediante Dijkstra Multidestino (calcular_estrategia_cercania_multidestino)

Seguir el orden administrativo en que fueron cargados los pedidos suele obligar al operario a ir y venir varias veces de una punta a otra del almacén. Para optimizar el recorrido sin recurrir a solvers exactos del Problema del Viajante (TSP) ni metaheurísticas prohibidas por las restricciones del proyecto, implementamos la estrategia del **vecino más cercano sobre el dígrafo mediante Dijkstra Multidestino** (calcular_ruta_dijkstra_multidestino):

- Todas las celdas de acceso requeridas por la wave se cargan inicialmente en un conjunto hash de objetivos pendientes P = {d_1, d_2,..., d_k}.
- En lugar de ejecutar k búsquedas de Dijkstra individuales desde p_{actual} hacia cada candidato $d \in P$ para ver cuál está más cerca (lo cual multiplicaría el costo por k en cada parada), ejecutamos **una única expansión de Dijkstra** desde p_{actual} pasando el conjunto completo P como condición de parada (if nodo_actual in set_destinos:).

- **Justificación algorítmica:** Dado que _MinHeapRuteo extrae los vértices en orden monótono no decreciente de distancia real desde p_{actual}, el primer nodo perteneciente a P que es extraído de la cola de prioridad es, por demostración directa de la propiedad de Dijkstra, el producto pendiente más cercano en el dígrafo.
- En ese mismo instante se detiene la búsqueda (early exit), se reconstruye el tramo hasta ese destino d^*, se elimina d^* del conjunto de pendientes (P ← P \ {d*}) en $\Theta(1)$, se actualiza $p_{actual} \leftarrow d^*$ y se repite el proceso hasta vaciar P, cerrando con el tramo de regreso a la zona de despacho.

Eliminación Perezosa en el Montículo de Dijkstra: Cuando durante la relajación de aristas se descubre un camino más corto hacia un vecino v que ya estaba dentro de _MinHeapRuteo con una distancia mayor, en lugar de buscar y modificar la clave dentro del arreglo del montículo, se inserta el nuevo par (nueva_distancia, nuevos_giros, v) directamente en $O(\log V)$. Al extraer un elemento en el ciclo principal, la condición if (dist_actual, giros_actual) > mejor_costo[nodo_actual]: continue descarta en $\Theta(1)$ cualquier entrada obsoleta. Dado que en nuestra grilla ortogonal cada celda tiene a lo sumo 4 aristas salientes ($E \leq 4V$), esta técnica mantiene el código simple y logra un rendimiento práctico superior.

4. Gestión de Tráfico, Concurrencia y Evasión de Colisiones (TurnAndCollisionManager)

De acuerdo con las reglas de movimiento del almacén, múltiples pickers se desplazan simultáneamente turno a turno: dos operarios nunca pueden ocupar la misma celda en el mismo instante ni intercambiar sus posiciones en un mismo paso (cruzarse de frente). Si dos pickers coinciden en un pasillo, el conflicto debe resolverse mediante espera o recálculo por camino alternativo.

Implementamos esta coordinación espacio-temporal en TurnAndCollisionManager (turn_collision_manager.py) en conjunto con los métodos de desvío de Pathfinding:

1. **Prioridad Física y Protección de Operarios en Tarea (_esta_fijo_en_celda):** Cuando un picker llega a la celda de acceso de un producto, permanece detenido durante turnos_recogida = 4 turnos cargando la mercadería del rack (turnos_pausa > 0), o bien cumple turnos_busqueda al iniciar su ruta. En el ordenamiento por turno (pickers_ordenados), los operarios fijos en tarea reciben la máxima prioridad física para no ser desplazados de su estantería; luego se prioriza a quien lleva más turnos bloqueado (-p.turnos_bloqueado) y a quien está más próximo a finalizar su recorrido (len(p.ruta_asignada)).
2. **Estela de Separación sin Autobloqueo (estela):** Al abandonar una celda, el picker deja una reserva temporal de turnos_estela = 2 turnos para evitar que otro operario camine pegado detrás de él y genere embotellamientos en cadena. Al evaluar desvíos, el sistema filtra únicamente la estela_ajena (pid != p.id_picker), permitiendo que un operario pueda retroceder sobre sus propios pasos en un pasillo de doble sentido si necesita maniobrar.
3. **Detección y Resolución en Tres Pasadas (validar_movimiento_simultaneo):**

- **Colisión en celda destino ($sig_{p1} = sig_{p2}$):** Si el operario cedente recién encuentra el bloqueo (turnos_bloqueado < max_turnos_bloqueo_recalculo), inserta su celda actual al inicio de ruta_asignada para **esperar 1 turno**. Si supera el umbral de espera (2 turnos), invoca a recalcular_evitando_obstaculo: este método identifica la próxima parada de recolección pendiente con proximo_objetivo_en_ruta(picker), bloquea las celdas ocupadas en el radio inmediato (dist_actual < 2) y ejecuta Dijkstra buscando un pasillo alternativo, empalmando luego el resto de las paradas originales (_empalmar_resto_ruta) para no omitir ningún producto intermedio.
- **Cruce frontal de aristas ($sig_{p1} = pos_{p2} \wedge sig_{p2} = pos_{p1}$):** En un choque de frente, esperar en el lugar no resuelve el bloqueo porque ambos operarios se obstruyen mutuamente. El sistema ejecuta _buscar_mejor_desvio_lateral, que evalúa todas las celdas ortogonales libres alrededor del operario cedente mediante Pathfinding.desviar_por (bloqueando la celda en conflicto para evitar que el camino recalculado rebote inmediatamente hacia atrás) y selecciona la maniobra lateral que ofrece la ruta más corta hacia su próximo objetivo.
- **Corrimiento de Pickers Libres y Descarga Adyacente:** Si un operario sin wave asignada obstruye el paso durante 2 turnos, se desplaza automáticamente a una celda vecina libre. Asimismo, si un picker que ya recolectó todos sus productos encuentra obstruida la última celda pero ya está en un nodo adyacente a su misma zona de despacho, descarga desde allí sin demorar la tanda.

5. Análisis del Comportamiento Estructural del Almacén

5.1. ¿Qué ocurre si todos los pasillos tienen el mismo largo?

En el diseño generado por nuestro sistema (y en los almacenes logísticos reales tipo grilla), los pasillos verticales de una misma fila comparten exactamente el mismo largo (por ejemplo, ALTO_RACK = 6 celdas flanqueadas por pasillos transversales de doble sentido en ambos extremos):

1. **Simetría de Costos y Degeneración de Caminos Mínimos:** Cuando todos los pasillos tienen igual longitud, cruzar de un extremo transversal al otro cuesta exactamente la misma cantidad de pasos sin importar qué pasillo vertical habilitado (del mismo sentido) se utilice. Esto genera una alta simetría en el grafo: existen múltiples caminos distintos con idéntica distancia mínima $\delta(u \to v)$.
2. **Importancia del Desempate Determinista:** Ante esta igualdad de costos, un Dijkstra estándar podría alternar arbitrariamente entre pasillos o cambiar de carril en cada fila. Gracias al ordenamiento por menor cantidad de giros en _MinHeapRuteo y al desempate determinista por coordenadas, el algoritmo mantiene al operario circulando por el mismo corredor longitudinal mientras sea posible.
3. **Penalización Uniforme de los Pasillos Unidireccionales:** Si un picker se encuentra en un pasillo vertical de sentido Norte (N) de largo L y debe recoger un producto ubicado una celda más atrás de su posición actual, la restricción de sentido

único lo obliga a terminar de subir el pasillo, tomar el transversal superior, bajar por el pasillo contiguo de sentido Sur (S) y volver a subir, sumando un ciclo fijo proporcional a $2L$. Por ello, ordenar las paradas con la Estrategia B (Cercanía) evita pasar de largo productos dentro de un mismo pasillo unidireccional, ahorrando vueltas completas de longitud $2L$.

5.2. ¿Cómo cambia el comportamiento cuando aumenta el tamaño del Warehouse?

A medida que aumentan las dimensiones del almacén ($ancho_x \times largo_y$), crece linealmente el número de celdas transitables V y de aristas dirigidas E:

1. **Escalabilidad Cuasi-Lineal del Ruteo:** Como cada celda de pasillo tiene grado de salida acotado ($deg^{+}(v) \leq 4$), el dígrafo es **ralo** ($E \leq 4V \Rightarrow E = \Theta(V)$). Por lo tanto, el costo de cada búsqueda de Dijkstra crece a razón de $O(V \log V)$. En las pruebas experimentales (variando de 6 a 160 estantes, es decir, de 216 a 5.476 nodos de pasillo), el tiempo de ejecución crece de manera suave y predecible sin sufrir explosión combinatoria.
2. **Aumento de la Brecha a favor de la Estrategia de Cercanía:** En un almacén pequeño (18×16), un orden subóptimo de paradas penaliza con pocos pasos adicionales. En cambio, cuando el almacén escala a tamaños grandes (60×40 o más), cruzar el mapa de punta a punta por seguir el orden de los pedidos multiplica las distancias y las expansiones del montículo, haciendo que la ventaja de **Dijkstra Multidestino (Cercanía)** sea cada vez más pronunciada tanto en distancia caminada como en tiempo de cómputo.

6. Análisis de Complejidad Temporal y Espacial (Sección 20)

Definimos las variables del módulo de ruteo como: V la cantidad de celdas de pasillo en el dígrafo, E la cantidad de aristas dirigidas (cumpliéndose $E \leq 4V = \Theta(V)$), K la cantidad de ubicaciones de recolección asignadas al picker en una wave, U_p la cantidad de celdas de acceso únicas con productos en el catálogo y F la cantidad de pickers activos en un turno.

Proceso / Método	Mejor Caso	Peor Caso	Espacio	Justificación Algorítmica
Inserción / Extracción en _MinHeapRuteo	$\Theta(1)$	$O(logV)$	$O(V)$	En el mejor caso el elemento queda en su hoja al insertar; en el peor sube o se hunde la altura

				completa del árbol binario $\lfloor log_2 V \rfloor$
Dijkstra Punto a Punto (calcular_ruta_dijkstra)	$\Theta(1)$	O((V + E) logV) = O(V logV)	$O(V)$	Mejor caso cuando origen == destino (corte inmediato). Peor caso cuando explora toda la componente conexa antes de alcanzar el destino.
Dijkstra Multidestino (calcular_ruta_dijkstra_multidestino)	$\Theta(1)$	O((V + E) logV) = O(V logV)	$O(V)$	Verifica pertenencia al conjunto set_destinos en $\Theta(1)$ y detiene la expansión apenas extrae el primer objetivo pendiente más cercano.
Estrategia A: Orden de Pedidos (calcular_estrategia_orden_pedidos)	$\Theta(K)$	$O(K \cdot V \log V)$	$O(V)$	Ejecuta a lo sumo $K + 1$ búsquedas de Dijkstra punto a punto (una por cada parada más el regreso a la zona de despacho).
Estrategia B: Cercanía (calcular_estrategia_cercania_multidestino)	$\Theta(K)$	$O(K \cdot V \log V)$	$O(V)$	Ejecuta $\| P \| \leq K$ expansiones de Dijkstra Multidestino (con corte temprano

				en el vecino más cercano) más 1 retorno a despacho.
Control de Tráfico y Evasión por Turno (validar_movimiento_simultaneo)	$\Theta(F^2)$	$O(F^2 + F \cdot V \log V)$	$O(V +$	Si no hay desvíos, compara pares y reservas en $\Theta(F^2)$. Si un picker recalcula ruta evitando obstáculos, ejecuta Dijkstra en $O(V \log V)$.

7. Comparación Experimental, Trade-offs y Modificaciones para un Entorno Real

7.1. Comparación de Estrategias: ¿Cuál recorre menor distancia y cuál tarda menos tiempo?

Al ejecutar ambas estrategias sobre exactamente las mismas waves, el mismo estado inicial y la misma cantidad de pickers en el escenario grande (escenario_grande.json, 60×40 celdas, 170 productos, 40 pedidos y 4 pickers), obtuvimos los siguientes resultados experimentales en el comparador del sistema:

Métrica Evaluada	**Estrategia A: Orden de Pedidos**	**Estrategia B: Cercanía (Dijkstra Multidestino)**	**Diferencia / Mejora**
Distancia total recorrida (pasos)	5.110	3.033	-40,6% pasos
Nodos visitados (celdas únicas pisadas)	3.606	2.824	-21,7%

Aristas recorridas	5.110	3.033	-40,6%
Nodos explorados (únicos)	27.849	25.477	-8,5%
Nodos expandidos (suma de búsquedas)	117.928	56.599	-52,0% expansiones
Tiempo de cálculo de rutas (ms)	$642,42ms$	$302,79ms$	-52,9% tiempo CPU
Turnos simulados hasta terminar	1.562	1.091	-30,2% turnos
Esperas por colisión / Desvíos	$59esperas/13desvíos$	$36esperas/5desvíos$	-39% conflictos

- **¿Cuál presenta menor distancia recorrida?:** La **Estrategia B (Cercanía)** presenta sistemáticamente una distancia recorrida mucho menor (un **40,6% menos** en el escenario grande: 3.033 pasos frente a 5.110). Esto se debe a que agrupa la recolección por proximidad espacial, barriendo los productos cercanos dentro de un mismo corredor antes de desplazarse a otro sector del almacén, lo que a su vez reduce las colisiones entre operarios (de 59 a 36 esperas).
- **¿Cuál presenta menor tiempo de ejecución?:** Gracias a nuestra implementación mediante **Dijkstra Multidestina**, la **Estrategia B (Cercanía)** también presenta **menor tiempo de ejecución** ($302,79ms$ frente a $642,42ms$, más del doble de rápida). La razón algorítmica es clara: en la Estrategia A, si el siguiente producto de la lista está en el otro extremo del almacén, Dijkstra debe expandir una gran porción del dígrafo en cada tramo (117.928 nodos expandidos en total); en cambio, en la Estrategia B cada búsqueda se detiene apenas toca el producto pendiente más

próximo, formando "esferas" de exploración pequeñas y locales que reducen los nodos expandidos a menos de la mitad (56.599).

7.2. ¿Existe una estrategia mejor en todos los escenarios y qué trade-offs aparecen?

- **No existe una estrategia que garantice el óptimo global en el 100% de las topologías posibles**: Dado que ordenar las paradas de una wave equivale al Problema del Viajante Asimétrico sobre un dígrafo (ATSP, que es NP-Difícil), la heurística del vecino más cercano (Estrategia B) toma decisiones localmente óptimas en cada paso. En escenarios patológicos muy específicos —por ejemplo, si el operario tiene una hilera de productos levemente más cercanos hacia el extremo opuesto a la zona de despacho y deja aislado un único producto cerca del inicio—, la Estrategia B puede dejar ese producto aislado para el final y hacer un tramo de retorno más largo que un orden preestablecido que ya venía regresando hacia el despacho. Sin embargo, en la totalidad de los escenarios logísticos estándar evaluados (tanto con distribución aleatoria, por rack o cercana al despacho), **Cercanía supera ampliamente a Orden de pedidos**.
- **Trade-offs entre tiempo de ejecución y calidad del recorrido**:
    1. Ruteo constructivo veloz vs. Optimización combinatoria exacta: Evaluar todas las permutaciones posibles de K productos tendría complejidad factorial $O(K!)$, volviéndose intratable para $K = 15$ o 20. Nuestro enfoque con Dijkstra Multidestino sacrifica una mínima fracción de optimalidad teórica global a cambio de obtener recorridos un ~40% más cortos que la línea base en apenas unos pocos milisegundos ($O(K \cdot V \log V)$).
    2. Espera pasiva vs. Recálculo dinámico ante colisiones: En TurnAndCollisionManager, recalcular la ruta con Dijkstra en el primer instante en que dos pickers se encuentran provocaría desvíos largos e innecesarios por bloqueos que duran apenas 1 turno. Por ello calibramos un umbral (max_turnos_bloqueo_recalculo = 2): esperar hasta 2 turnos tiene costo computacional $\Theta(1)$ y resuelve cruces pasajeros, reservando el recálculo completo $O(V \log V)$ solo para obstrucciones persistentes (como un operario detenido recogiendo varios artículos en un rack).

7.3. Modificaciones para utilizar el sistema en un Warehouse real

Para llevar este motor de ruteo y control de tráfico a un centro de distribución industrial real, incorporaríamos cuatro extensiones directas sobre nuestra arquitectura:

1. **Pesos de Arista Dinámicos y Tiempos Físicos Diferenciados:** En lugar de asumir costo uniforme $w = 1$ para todo movimiento, ponderar las aristas del dígrafo según la velocidad real del autoelevador en rectas frente a la desaceleración en curvas/intersecciones, y asignar un tiempo de recolección en estantería proporcional a la altura del nivel z (ya que elevar las horquillas a $z = 5$ toma mucho más tiempo que recoger a nivel de piso $z = 1$).
2. **Planificación Cooperativa Espacio-Temporal:** Reemplazar la evasión reactiva de corto alcance por una reserva completa de ventanas de tiempo durante la planificación inicial, de modo que las rutas de toda la flota se calculen coordinadamente sin conflictos en los próximos T segundos.

3. **Recálculo Reactivo ante Eventos en Tiempo Real (Re-batching Dinámico):** Permitir que si ingresa un pedido urgente de prioridad 5 mientras un picker ya está caminando por ese mismo pasillo con capacidad libre en su carro, el sistema le inyecte esa parada adicional en caliente recalculando su Dijkstra Multidestino desde su coordenada actual.
4. **Restricciones de Apilamiento y Fragilidad (3D Bin Packing):** Incorporar junto al límite de peso (capacidad_carga_kg) el volumen tridimensional de las cajas y su fragilidad, ordenando el recorrido para recolectar primero los productos pesados/voluminosos (que van en la base del pallet) y dejando los productos frágiles para el final del recorrido antes de volver a despacho.

Capítulo 3: Generación de Waves y Asignación de Pickers

1. Arquitectura del Módulo y Estructuras de Datos

Para cumplir con la separación entre la lógica algorítmica y la interfaz visual, dividimos el subsistema de waves en tres componentes coordinados bajo un patrón Fachada (*Facade*)[1]: `WaveManager` actúa como punto de entrada único, delegando la agrupación de pedidos en `WaveBuilder` y la distribución de operarios y cálculo de rutas en `WaveAssigner`.
Para garantizar que los pedidos de mayor urgencia (prioridad de 1 a 5, siendo 5 la más alta) sean atendidos primero, implementamos nuestra propia cola de prioridad mediante un Montículo Binario de Máximos (MaxHeapPedidos). En lugar de reordenar listas en cada iteración, esta estructura nos permite:

> El Patrón Fachada (Facade) es un patrón de diseño de software de tipo estructural que sirve para proporcionar una interfaz unificada, simple y de alto nivel a un subsistema complejo o a un conjunto de clases inter relacionadas.
>
> En términos sencillos: actúa como una "ventanilla única" que esconde toda la complejidad interna de varios componentes, permitiendo que el resto del sistema interactúe con ellos de forma limpia y directa sin necesidad de conocer sus detalles internos.

- **Construir el montículo inicial** a partir de los pedidos válidos utilizando el algoritmo de Floyd[2] (construir_monticulo) en tiempo lineal $\Theta(N)$.
- **Extraer el pedido más prioritario** (extraer_maximo) en tiempo $O(\log N)$ para usarlo como "semilla"[3] de una nueva wave. Para que el algoritmo sea 100% determinista, en caso de empate en prioridad desempatamos por orden alfabético del identificador del pedido (id_pedido).

- **Eliminar pedidos intermedios en $O(\log N)$:** Cuando un pedido que no está en la raíz es elegido por cercanía espacial para sumarse a la wave actual, necesitamos quitarlo del montículo sin hacer una búsqueda secuencial $O(N)$. Para lograrlo, mantuvimos un diccionario interno (posiciones_id) que mapea cada id_pedido con su índice exacto dentro del arreglo del heap, logrando ubicarlo en $\Theta(1)$ y reequilibrar el árbol en $O(\log N)$

El algoritmo de Floyd Es un algoritmo inventado por Robert Floyd que transforma un arreglo desordenado de N elementos en un montículo binario (*Max-Heap*) en **tiempo lineal** $\Theta(N)$.

- **Diferencia con el método ingenuo:** Si insertáramos los N elementos uno por uno usando insertar(), cada inserción tomaría $O(\log N)$, dando un total de $O(N \log N)$.
- **El enfoque de Floyd (Bottom-Up):**
  1. Se carga el arreglo con los N elementos tal como vienen.
  2. Las hojas del árbol (desde la posición $\lfloor N/2 \rfloor + 1$ hasta N) ya cumplen trivialmente la propiedad de montículo porque no tienen hijos.
  3. Se recorren los nodos internos en reversa (desde el último padre $\lfloor N/2 \rfloor$ hasta la raíz 0) aplicando la operación de hundimiento o tamizado (MAX-HEAPIFY).
- **Justificación matemática del tiempo lineal:** La mayoría de los nodos están en los niveles inferiores (cerca de las hojas) y se hunden a lo sumo 1 o 2 niveles; muy pocos nodos están en la cima para hundirse la altura completa h. La sumatoria de alturas ponderadas por la cantidad de nodos por nivel converge:

$$\sum_{h=0}^{|lnN|} N/2^{h+1} O(n) = \theta(N)$$

El procedimiento para construir un montículo binario en tiempo lineal se le atribuye formalmente a **Robert W. Floyd (1964)** bajo la rutina **`BUILD-MAX-HEAP(A)`**.

El punto de partida geográfico: Una ola vacía no tiene ubicación física ni referencia espacial. Al extraer el pedido más prioritario, este se convierte en la **semilla**: el primer pedido colocado dentro de la *wave*.

El centro de atracción espacial: A partir de ese momento, los productos del pedido semilla definen la zona del almacén donde operará esa ola[1]. Todos los pedidos siguientes que intenten sumarse a la *wave* se seleccionarán en función de qué tan cerca estén de esta semilla (mediante el Costo Marginal Promedio o *Average Linkage*)

Además, en el modelo de entidades (entities.py) incorporamos **memorización (caché interna)** dentro de la clase Pedido (_cache_unidades, _cache_peso y _cache_xyz). Como durante el armado de waves un mismo pedido candidato es evaluado muchas veces contra distintas olas, calcular su peso y sus ubicaciones una sola vez y reutilizar el resultado en $\Theta(1)$ redujo drásticamente el tiempo de CPU.

2. Validación de Restricciones y Reserva Virtual de Inventario

Antes de armar las waves, el sistema realiza un filtrado inicial (`validar_pedidos_entrada`) para detectar pedidos que son físicamente imposibles de preparar en el escenario cargado. Si un pedido solicita más unidades que el tope `wave_max_productos`, supera por sí solo la capacidad de carga del picker más grande del almacén, pide más stock del que existe en los racks o tiene productos en pasillos desconectados, el sistema lo marca como `IRREALIZABLE` y registra el motivo para informar al usuario sin bloquear el resto de la simulación.
Durante la construcción de cada wave, controlamos simultáneamente cinco restricciones operativas:

1. No superar la cantidad máxima de pedidos por wave (wave_max_pedidos).
2. No superar la cantidad máxima de unidades por wave (wave_max_productos).
3. **Compromiso virtual de stock:** Al generar las waves, los pickers todavía no caminaron por el almacén, por lo que sería incorrecto descontar el stock real de las estanterías en ese instante. Sin embargo, tampoco podemos permitir que dos waves distintas cuenten con el mismo stock físico si este es escaso. Lo resolvimos mediante un diccionario de reserva temporal (stock_comprometido_virtual) que bloquea virtualmente las unidades a medida que los pedidos entran en una wave. El descuento físico real recién se ejecuta cuando el picker llega a la zona de despacho y descarga los productos.
4. **Capacidad y tipo de Zona de Despacho:** Filtramos únicamente las zonas asociadas a puertas de salida (OUTBOUND o AMBOS). Al iniciar una wave, elegimos la zona con capacidad disponible más cercana al centroide[4] de los productos del pedido semilla. Para evitar sobrecargar la zona cuando hay varias waves activas, implementamos en ZonaDespacho un doble control: sumamos tanto el peso que ya está depositado físicamente (peso_actual_kg) como el peso reservado por waves en curso (peso_comprometido_waves_kg), garantizando que nunca se exceda peso_max_kg.
5. **Factibilidad de carga en la flota de Pickers:** Mediante el método es_repartible_en_equipo_pickers, verificamos con una heurística voraz (*Best-Fit Decreasing*)[5] que los pedidos acumulados en la wave puedan ser transportados por al menos un picker (o un equipo de hasta max_pickers_por_wave operarios) sin que ninguno supere su capacidad_carga_kg.

Matemáticamente, el **centroide bidimensional** (c_x, c_y) de un pedido es el promedio aritmético de las posiciones en planta (x, y) de todos los productos que lo componen

$$Cx = 1/|U| \sum_{x \in U} x,\ Cy = 1/|U| \sum_{y \in Y} y$$

Donde U es el conjunto de ubicaciones únicas en los racks donde se encuentran los artículos solicitados en dicho pedido. Este punto representa la **"posición geográfica promedio"** donde se concentra la recolección de esa orden de compra

La heurística voraz Best-Fit Decreasing (BFD) es uno de los algoritmos de aproximación más reconocidos y eficientes para resolver problemas de empaquetamiento (Bin Packing Problem) y partición de recursos con capacidades limitadas
El problema clásico de *Bin Packing* consiste en guardar un conjunto de objetos de diferentes pesos o tamaños dentro de la menor cantidad posible de recipientes (*bins*) de capacidad fija. Dado que este problema es NP-Duro (no existe un algoritmo polinomial exacto a menos que $P = NP$), se emplean heurísticas voraces (*greedy*)
BFD combina dos principios fundamentales:

1. **Decreasing (Ordenamiento Decreciente):**
    - Antes de empaquetar, se ordenan todos los elementos de **mayor a menor** según su tamaño o peso.
    - *Intuición voraz:* Los objetos más grandes y pesados son los más difíciles de acomodar; si se dejan para el final, es muy probable que no quepan en los espacios sobrantes y fuercen a abrir nuevos recipientes. Ubicarlos primero asegura que los objetos pequeños actúen como "relleno" en los huecos residuales.
2. **Best-Fit (Mejor Ajuste):**
    - Al momento de asignar un elemento, se examinan todos los recipientes que tengan capacidad disponible para alojarlo.
    - Se elige aquel recipiente que represente el **"mejor ajuste"**: aquel donde el espacio libre restante después de colocar el objeto sea el más pequeño posible (es decir, el que quede más ajustado o lleno), o aquel que balancee la carga de forma óptima.
    - Si no cabe en ningún recipiente abierto, se abre uno nuevo.

Rescate de Waves Pesadas (Caso 4 de Asignación)
Si una wave ya cerrada resulta tener un peso acumulado superior a la capacidad máxima de cualquier picker individual del almacén, el sistema ejecuta BFD para determinar cuál es el equipo mínimo de operarios (entre 2 y `max_pickers_por_wave`) necesario para transportar y dividir los pedidos de la ola colaborativamente

3. Cálculo de Distancias entre Ubicaciones en el Dígrafo

Para que el armado de waves utilice distancias reales sin tener que correr el algoritmo de Dijkstra miles de veces desde cero, precomputamos al inicio una matriz de distancias dirigidas (matriz_distancias) entre todas las celdas de pasillo que dan acceso a los productos del catálogo. Así, durante el armado de waves, consultar la distancia real en el dígrafo entre dos productos toma tiempo constante $\Theta(1)$.

Como en un grafo dirigido ir del producto u al producto p no mide lo mismo que ir de p a u ($\delta(u \to p) \neq \delta(p \to u)$), y en la etapa de agrupamiento todavía no sabemos en qué orden exacto los visitará el picker, definimos la **distancia de enlace** entre dos ubicaciones como el mínimo de ambos sentidos:

$$denlace(p,u) = (\delta digrafo(u \to p) \neq \delta digrafo(p \to u))$$

4. Estrategias de Armado de Waves y Modo Automático

El algoritmo de construcción (generar_waves) funciona de manera iterativa: mientras queden pedidos en el MaxHeap, extrae la raíz (el pedido de mayor prioridad disponible)

para abrir una nueva wave. Luego, busca en el montículo todos los pedidos pendientes cuya cantidad de artículos quepa en el espacio libre de la wave (obtener_candidatos_factibles).

Para decidir qué pedido candidato conviene agregar en cada paso, diseñamos e implementamos **tres estrategias de agrupamiento distintas** (además de un selector automático). En todas ellas, el candidato ganador es el que minimiza la tupla $(Score, -\ prioridad, id_pedido)$, lo que asegura que ante igual puntaje espacial siempre se prefiera al pedido de mayor prioridad:

1. **Cercanía en el Dígrafo (CERCANIA — Costo Marginal Promedio):** Para cada pedido candidato, calculamos qué tan lejos están sus productos respecto a los productos que ya forman parte de la wave. En lugar de mirar solo el punto más cercano (lo que podría encadenar pedidos cada vez más alejados), promediamos las distancias mínimas de todos los ítems del candidato hacia la wave. Además, si un producto se encuentra en una coordenada (x, y, z) que aún no fue visitada por la wave, sumamos una penalización proporcional a la altura z; si otro pedido de la misma wave ya visitaba ese mismo estante (x, y, z), el costo marginal es 0.
2. **Afinidad por Pasillo (PASILLO):**
   Identificamos desde qué pasillos se accede a los productos de la wave actual y contamos cuántos **pasillos nuevos** obligaría a visitar cada pedido candidato: $ScorePASILLO(C, W) = |Pasillos(C) \backslash Pasillos(W)|$
   Esta estrategia busca concentrar el trabajo del picker dentro de la menor cantidad posible de corredores.
3. **Solo Prioridad (PRIORIDAD):** Asigna $Score = 0$ a todos los candidatos factibles, por lo que agrupa los pedidos siguiendo estrictamente su nivel de urgencia (y desempata por ID) sin importar en qué parte del almacén estén ubicados sus productos. Incluimos esta estrategia como línea base (*baseline*) para medir cuánto recorrido ahorran las heurísticas espaciales.
4. **Selección Automática (AUTO) y Comparador de Armado:** Dado que ninguna estrategia es universalmente superior en todos los diseños de almacén, implementamos en EvaluadorEstrategias el método comparar_armado. Este método clona el estado del simulador antes de armar las waves, ejecuta de forma aislada las tres estrategias completas (armado + ruteo + simulación de turnos) y elige automáticamente aquella que logra la menor distancia total recorrida y menor cantidad de turnos.

5. Asignación de Pickers y Reparto Colaborativo de Carga

Una vez creadas las waves, el módulo WaveAssigner se encarga de asignarlas a los operarios libres (id_wave_actual is None) y planificar sus recorridos desde y hacia la zona de despacho.

5.1. Selección de Pickers por Cercanía Real

Para decidir qué picker debe tomar una wave, calculamos la distancia real en el dígrafo desde la posición actual de cada picker libre hasta el producto más cercano de la wave (`calcular_distancia_picker_a_wave`), ordenando a los candidatos por menor distancia, mayor capacidad de carga en kilos y menor ID.

5.2. Los 5 Casos de Asignación Operativa

Al momento de asignar operarios a una wave seleccionada, contemplamos cinco escenarios posibles:

- **Caso 1 (Asignación Individual Directa):** Si se configura 1 picker por wave y el operario más cercano tiene capacidad de carga suficiente ($capacidad_carga_kg \geq W.peso_total_kg$), toma la wave completa. Si el más cercano no soporta el peso, el algoritmo busca al siguiente picker libre más cercano que sí pueda cargarla.
- **Caso 2 (Asignación Colaborativa Solicitada):** Si el usuario solicita $m > 1$ pickers por wave, se toman los m pickers libres más cercanos y se verifica mediante equipo_puede_cargar que puedan repartirse los pedidos sin exceder sus capacidades individuales.
- **Caso 3 (Aceleración Colaborativa Automática):** Si el usuario manda a ejecutar **una única wave** y en ese momento hay varios pickers libres ociosos en el almacén, el sistema aprovecha automáticamente esa fuerza de trabajo asignando un equipo de hasta $min(|pickers_libres|, |W.pedidos|, max_pickers_por_wave)$ operarios para terminar la wave en menos turnos.
- **Caso 4 (Rescate Multi-Picker por Exceso de Peso):** Si una wave acumula un peso total que supera la capacidad de cualquier picker individual disponible, pero es posible dividir sus pedidos entre 2 o más pickers libres (es_repartible_en_equipo_pickers), se conforma automáticamente un equipo colaborativo para que la wave no quede bloqueada.
- **Caso 5 (Cola de Espera Reactiva):** Si en el turno actual no hay pickers libres (o los que están libres no alcanzan a cubrir el peso de la wave), la wave se coloca en la lista en_espera de la tanda actual. Apenas un picker en circulación llega a la zona de despacho y descarga su mercadería, el simulador detecta el evento y le asigna inmediatamente la wave que estaba esperando.

5.3. Reparto de Pedidos por Centroide 2D y Despacho sin Bloqueos

Cuando una wave es atendida por un equipo de varios pickers (Wave.asignar_equipo_pickers), no repartimos los pedidos al azar. Para cada pedido de la wave, calculamos el **centroide geométrico** (c_x, c_y) de sus productos y se lo asignamos al picker del equipo (que aún tenga capacidad de carga libre) cuya posición actual esté más cerca de ese centroide. De esta forma, cada operario recolecta el subconjunto de pedidos más próximo a su sector inicial, evitando que crucen todo el almacén innecesariamente. Finalmente, al regresar a descargar a la zona de despacho, el método elegir_celda_despacho busca todas las celdas de pasillo adyacentes a la zona y le asigna a cada picker una celda libre distinta, priorizando aquellas que no bloqueen corredores de un solo carril.

6. Análisis de Complejidad Temporal y Espacial

Definimos las variables del problema como: N la cantidad de pedidos pendientes, I el promedio de ítems por pedido, K la capacidad máxima de unidades por wave (wave_max_productos), P_W el máximo de pedidos por wave (wave_max_pedidos), U_P la

cantidad de ubicaciones (x, y) con productos en el catálogo, F el total de pickers y V, E los vértices y aristas del dígrafo de pasillos.

Proceso / Método	Mejor Caso	Peor Caso	Espacio	Justificación Algorítmica
Precomputación de Distancias (`matriz_distancias`)	$\Omega(U_p(V + E))$	$O(U_p(V + E)logV)$	$O(U_P{}^2)$	Se ejecuta una sola vez al cargar el almacén corriendo Dijkstra desde cada ubicación de producto
Validación Inicial y `MaxHeap`	$\Omega(N)$	$O(N * I + N)$	$O(N)$	En la primera lectura calcula y guarda en caché el peso y ubicaciones de cada pedido ($O(N \cdot I)$) y arma el montículo con Floyd en $\Theta(N)$.
Costo Marginal de 1 Candidato (`CERCANIA`)	$\Omega(1)$	$O(K^2)$	$O(1)$	En el mejor caso los productos ya están en la wave; en el peor compara a lo sumo K productos del candidato contra K de la wave en $\Theta(1)$ por par.
Generación Total (`generar_waves`)	$\Omega(NlogN)$	$O(K^2 * N^2)$	$O(N)$	El mejor caso ocurre en PRIORIDAD o cuando casi ningún candidato cabe (solo extracciones del Heap). El peor caso evalúa hasta N candidatos por cada inserción.
Asignación de Pickers y Rutas (`asignar_y_ejecutar`)	$\Omega(W_{sel} * FlogF)$	$O((F + W_{sel} * K)(V + E)logV)$	$O(F + W_{sel} * K + V)$	Calcula distancias desde los F pickers libres, ordena candidatos en $O(F \log F)$, reparte por centroide en $O(P_W \cdot F)$ y traza la ruta con Dijkstra.

7. Discusión de Trade-offs y Conclusiones del Módulo

Al contrastar las estrategias implementadas en los distintos escenarios de prueba, observamos un *trade-off* (compromiso) muy claro entre el **tiempo de cómputo invertido al armar la wave** y la **calidad de los recorridos resultantes**:

- **Velocidad de cómputo vs. Distancia caminada:** La estrategia PRIORIDAD es la más veloz en tiempo de procesador (fracciones de milisegundo) porque no realiza cálculos geométricos ni consulta la matriz de distancias; simplemente vacía el MaxHeap en orden de urgencia. Sin embargo, al agrupar pedidos cuyos productos están en extremos opuestos del almacén, obliga a los pickers a recorrer distancias mucho mayores y a ingresar a casi todos los pasillos.
- **Beneficio del agrupamiento espacial (CERCANIA y PASILLO):** Aunque evaluar el costo marginal promedio o la intersección de pasillos requiere analizar más candidatos en cada iteración (unos pocos milisegundos adicionales de CPU), las waves resultantes quedan altamente concentradas en sectores específicos del almacén. En la simulación interactiva, esto se traduce en una reducción significativa de la distancia total recorrida, menos congestión y esperas entre pickers dentro de los pasillos de mano única, y una menor cantidad de turnos totales para completar la tanda de pedidos.

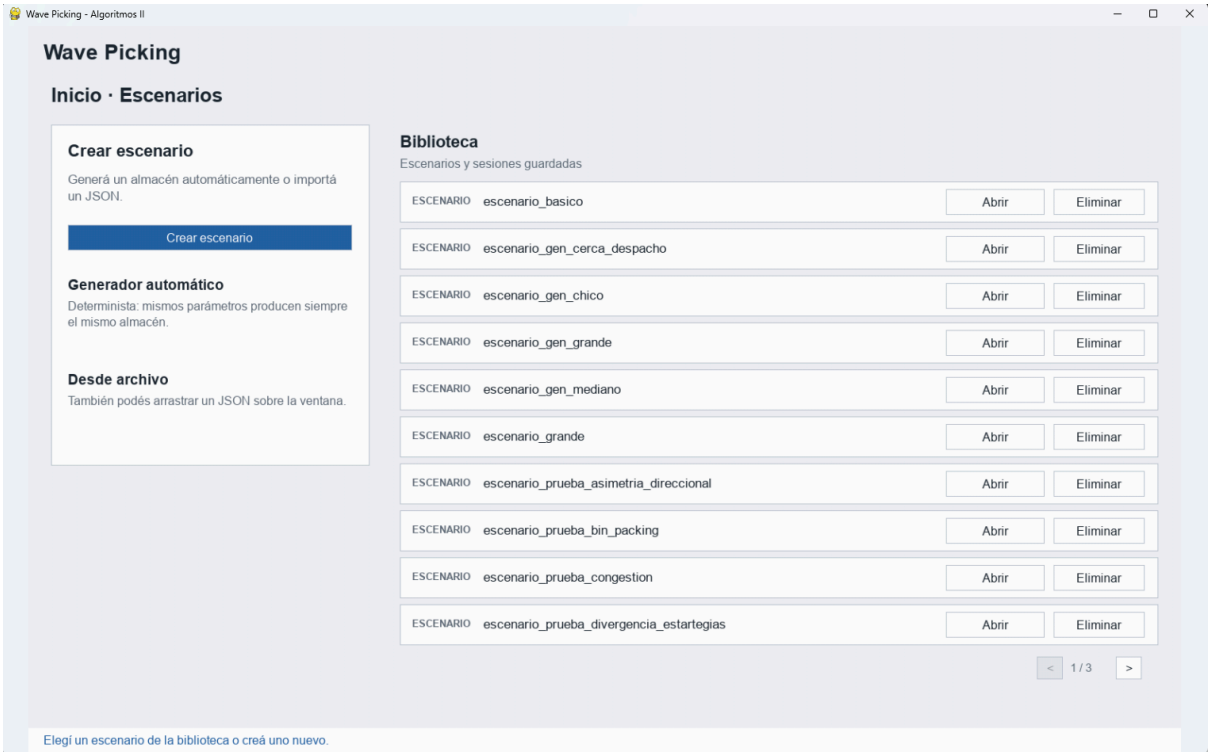
Capítulo 4: Implementación visual del proyecto

1. Arquitectura del Módulo y Estructuras de Datos

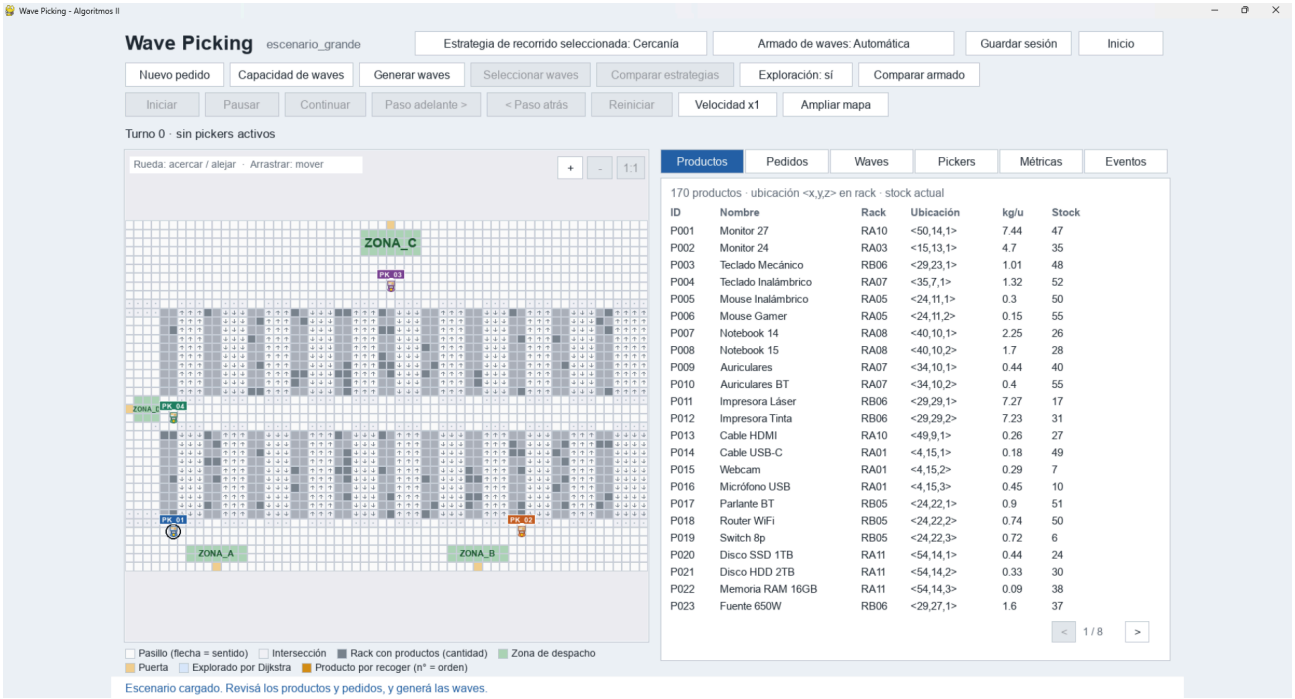
La interfaz gráfica, implementada en Python con Pygame, permite cargar el almacén, preparar una tanda de waves y observar cómo los pickers calculan sus rutas y recolectan los pedidos. La clase App (**pygame_engine.py**) coordina dos pantallas, la biblioteca de escenarios y la simulación. El objeto Simulador administra el estado del almacén, el stock y los turnos. La interfaz consulta sus resultados y traduce las acciones del usuario en llamadas al simulador.

input_handler.py interpreta el teclado y el mouse. **VistaMapa** dibuja el almacén y maneja el zoom y el arrastre. **PanelesVistas** y **DialogosVistas** agrupan las pestañas, formularios y comparaciones. **UIElements** aporta los componentes reutilizables, como botones y cuadros de texto. Estas tres clases se incorporan a **App** como mixins, es decir, clases que aportan métodos a otra clase mediante herencia sin constituir aplicaciones independientes. Biblioteca administra los archivos JSON de escenarios y sesiones.

La interfaz usa una lista de botones con su rectángulo, acción y estado de habilitación, un conjunto para los identificadores de waves marcadas y diccionarios indexados por coordenada o por identificador. Por ejemplo, productos_por_celda permite consultar cuántos productos hay en una celda de rack y los registros de animación guardan la posición anterior y la nueva de cada picker.



Interfaz de la biblioteca de escenarios.



Interfaz de la simulación del warehouse.

2. Validaciones Antes, Durante y Después de la Simulación

Antes. La biblioteca comprueba la estructura del JSON y delega al validador las reglas del escenario descritas en el capítulo 1. Si se importa un archivo y el almacén presenta advertencias, se informa el problema antes de incorporarlo a la biblioteca. Para crear un pedido se comprueban su prioridad, la existencia de los productos y las cantidades positivas. Al configurar las waves se validan los límites de pedidos, unidades y pickers.

Durante. El menú permite marcar waves pendientes y exige una selección y pickers libres para habilitar la asignación. El simulador verifica la capacidad disponible. Una vez iniciadas las waves, se mantiene visible la estrategia con la que se armaron y se bloquea su edición. Continuar se habilita cuando hay una tanda activa cuya reproducción ya comenzó y está pausada. La gestión de conflictos de movimiento corresponde al simulador y la vista muestra sus efectos en los pickers y en el registro de eventos.

Después. Al terminar la tanda se pueden consultar el estado de los pedidos, los turnos, los pasos efectivos, las esperas y los desvíos. El guardado conserva el escenario base y un registro de acciones en JSON. Al abrir una sesión se verifica su versión y se reconstruye el estado reproduciendo las acciones permitidas. El archivo se escribe primero con un nombre temporal y se reemplaza al completarse la escritura.

3. Estructuras Internas y Algoritmos de Visualización

El bucle de eventos separa la entrada, la actualización y el dibujo. La pantalla lógica se redimensiona para ajustarse a la ventana y se aplica la transformación inversa al clic. En el mapa se invierte el eje vertical, ya que la coordenada y del almacén crece hacia arriba y la de Pygame hacia abajo. Con el tamaño de la celda y el desplazamiento de la vista se obtiene la celda seleccionada.

Para animar el movimiento se interpola entre dos posiciones válidas, es decir, se dibujan posiciones intermedias en función del tiempo transcurrido. El stock, las colisiones y las recogidas se actualizan por turnos en el simulador. Para representar Dijkstra se usa el orden de exploración registrado por el backend. Se eliminan repeticiones con un conjunto sin perder el orden de la primera visita, se muestran progresivamente las celdas exploradas y luego se dibuja la ruta. La visualización del armado usa otra traza con el pedido raíz, los candidatos, el pedido incorporado y el cierre de la wave.

El fondo del mapa se guarda en una caché, una copia temporal de una imagen ya dibujada que puede reutilizarse mientras se mantiene la misma vista. Se reconstruye cuando cambian el escenario, la región de dibujo, el zoom, el desplazamiento o el plan de exploración mostrado. Los pickers y las rutas se dibujan encima. Para retroceder, el simulador conserva copias periódicas del estado y reproduce los turnos intermedios desde una copia anterior cuando hace falta.

4. Estrategias Implementadas

La interfaz presenta el armado y el recorrido como decisiones separadas. Para armar las waves ofrece **Cercanía en el dígrafo**, **Por pasillo** y **Solo prioridad**. Al abrir un escenario

sin una elección explícita aparece Automática. El simulador evalúa los tres criterios sobre copias del mismo escenario. Usa la menor distancia planificada como criterio principal y, en caso de empate, compara turnos, pasillos por wave y tiempo de armado, en ese orden. El botón muestra el criterio efectivo después de generar las waves.

Para recorrer las waves se puede elegir **Orden de pedidos** o **Cercanía**. La comparación de recorrido parte de las mismas waves y del mismo estado inicial. La comparación de armado conserva la estrategia de recorrido mientras cambia la forma de agrupar los pedidos. Ambos cuadros muestran resultados sin aplicar una selección al escenario. La pestaña Métricas distingue los pasos efectivos de la distancia del plan. Los experimentos por lotes se ejecutan desde consola y sus resultados se exportan por separado a CSV y, con openpyxl, a Excel.

5. Análisis de Complejidad Temporal y Espacial

Sean **B** el número de controles registrados, **C** las celdas visibles, **Z** las zonas de despacho, **P** los pickers y **R** las entradas de rutas, exploraciones y recorridos que la vista consulta o copia. Resolver un clic requiere **O(B)** comparaciones en el peor caso y **O(1)** si el primer control examinado lo recibe. La transformación de un punto de la ventana a una celda cuesta **O(1)**.

Construir el fondo cuesta **O(C + Z)** en operaciones sobre elementos, con **C** acotado por el ancho multiplicado por el largo. Si la vista sigue igual se reutiliza la imagen. Las capas dinámicas requieren **O(P + R)**, además del trabajo de dibujar los paneles. La primera deduplicación de una traza de **E** registros cuesta **O(E)** en tiempo y espacio auxiliar. Una tabla puede dibujar solo su página visible, aunque la preparación de todas sus filas sigue dependiendo del total de registros.

La vista ocupa **O(A + B + P + R)** de memoria para un cuadro, donde **A** es la cantidad de píxeles de las superficies de dibujo. A esto se suman el catálogo visual por celdas, las trazas retenidas y los estados del simulador. El costo de la comparación se estima sumando los costos de las corridas que ejecuta el backend, dos para recorrido y tres para armado. El tamaño del mapa y de las tandas afecta ese tiempo, incluso cuando la tabla que muestra los resultados tiene pocas filas.